A-Z Linear Algebra & Calculus for AI, Data Science and Machine Learning

Instructor:
Rashedul Alam Shakil
Founder of Study Mart
Founder of aiQuest Intelligence
Master in Data Science at FAU Erlangen
Automation Programmer at Siemens Energy, Germany

Regression Analysis in Machine Learning

Watch Tutorial

Regression analysis in machine learning is a **supervised learning technique** used to model the relationship between a **dependent variable (target)** and one or more **independent variables (features)**. The goal is to **predict continuous values** based on the input data.

Here's the gist:

- **Input:** Historical data with known outputs (e.g., stock price, house prices, temperatures).
- **Output:** A model that can predict future values for unseen inputs.

🔍 Key Concepts:

- **Dependent variable (Y):** What you're trying to predict (e.g., price of a house).
- **Independent variables (X):** The inputs or features (e.g., number of rooms, square footage).
- **Regression model:** A function that maps inputs to predicted outputs.

🛒 Scenario: Predicting House Prices

You work for a real estate company and want to build a machine learning model that predicts the **price of a house** based on its features.

Square Footage	Bedrooms	Bathrooms	Location	Price ($)
2000	3	2	Suburb	300,000
1500	2	1	City	350,000
2500	4	3	Suburb	400,000

- **Independent variables (Features):** Square Footage, Bedrooms, Bathrooms, Location
- **Dependent variable (Target):** Price ($)

📊 Scenario: Predicting Stock Prices for a Company

Let’s say you want to predict the **next day's stock price** for Apple based on historical trends.

Date	Open	High	Low	Close	Volume
2025-01-01	180.00	182.00	178.00	181.00	40,000,000
2025-01-02	181.50	183.00	180.00	182.50	35,000,000
2025-01-03	182.00	185.00	181.00	183.50	45,000,000

Algorithm Type	Description
❑ **Linear Regression**	Assumes a straight-line relationship between input(s) and output.
❑ **Multiple Linear Regression**	Like linear, but with multiple input features.
❑ **Polynomial Regression**	Fits a curved line to the data.
❑ **Ridge/Lasso Regression**	Linear regression with regularization to avoid overfitting.
❑ **Logistic Regression**	Despite the name, it's used for classification (not regression).
❑ **Decision Tree / Random Forest Regression**	Non-linear models based on tree structures.
❑ **Support Vector Regression (SVR)**	Uses SVM principles to predict continuous values.

Linear Regression in Machine Learning

Watch Tutorial

Say you want to predict the **price of a house** based on its **square footage**.

Square Footage (X)	Price ($Y)
1000	200000
1500	250000
2000	300000
2200 (Testing)	**?**

Price = (Slope × Square Footage) + Intercept

Price(y) = mx + c

Mean-Based Formula for OLS:

$$m = \frac{\sum(x - \bar{x})(y - \bar{y})}{\sum(x - \bar{x})^2}$$

$$c = \bar{y} - m\bar{x}$$

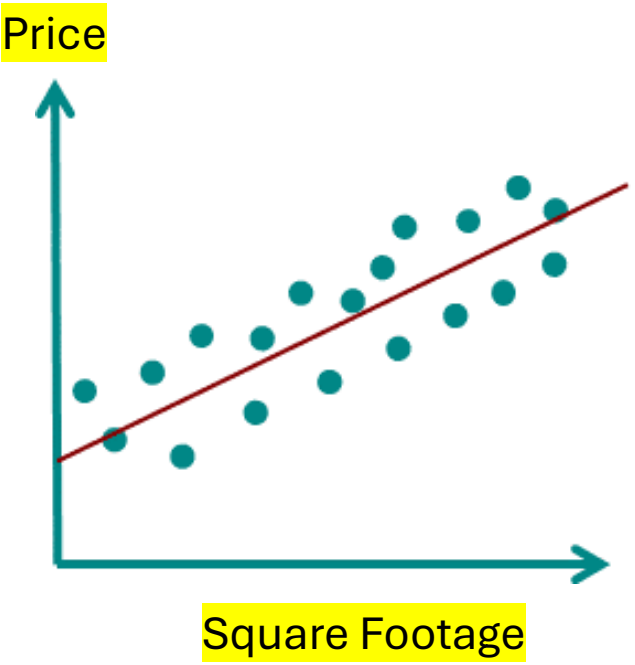
Price(y) = mx + c

Mean-Based Formula for OLS:

$$m = \frac{\sum(x - \bar{x})(y - \bar{y})}{\sum(x - \bar{x})^2}$$

$$c = \bar{y} - m\bar{x}$$

Price(y) = mx + c



Loss: Original_Price(y) - Price(y)

Mean-Based Formula for OLS:

$$m = \frac{\sum(x - \bar{x})(y - \bar{y})}{\sum(x - \bar{x})^2}$$

$$c = \bar{y} - m\bar{x}$$

Price(y) = mx + c

Loss: Original_Price(y) - Price(y)

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \quad \text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \quad \text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

How to compute m1, m2, m..., and c in multiple variable linear regression?

⚠️ The Problem with Mean-Based Formulas

Machine Learning

In simple linear regression (1 input), you can use:

$$m = \frac{\sum(x - \bar{x})(y - \bar{y})}{\sum(x - \bar{x})^2} \quad \text{and} \quad c = \bar{y} - m\bar{x}$$

- r_{12} = correlation between x_1 and x_2
- s_1, s_2 = standard deviations of x_1 and x_2

For two features: $$m_1 = \frac{\sum(x_1 - \bar{x}_1)(y - \bar{y}) - r_{12} \cdot s_1/s_2 \cdot \sum(x_2 - \bar{x}_2)(y - \bar{y})}{\sum(x_1 - \bar{x}_1)^2 \cdot (1 - r_{12}^2)}$$

For **multiple features**, things get messy:

- You'd need to calculate **partial correlations**, **standard deviations**, and **adjust for multicollinearity** (how features influence each other).
- The formulas for m1, m2 become **long and complex**.
- Doesn't scale well to 3, 5, 10, or 100 **features**.
- Easy to make mistakes when features are correlated.

It **does change** and gets **way more complex** due to the interaction between features.

What is the Solution?

Linear Regression

The 'Normal Equation' for Linear Regression

Use matrix form: $\mathbf{w} = (X^T X)^{-1} X^T y$

X is the matrix of inputs: $X = \begin{bmatrix} 1 & x_{11} & x_{12} \\ 1 & x_{21} & x_{22} \\ \vdots & \vdots & \vdots \\ 1 & x_{n1} & x_{n2} \end{bmatrix}$

- First column = all 1s (for the intercept c)
- Each row = one data point
- x_{ij} = value of the **j-th feature** for the **i-th example**

The **first column of 1's** is **not the value of c**. It's a **placeholder column** to **learn C** (the **intercept**) during matrix multiplication.

Linear Regression

The 'Normal Equation' for Linear Regression

Use matrix form: $\mathbf{w} = (X^T X)^{-1} X^T y$

X is the matrix of inputs: $X = \begin{bmatrix} 1 & x_{11} & x_{12} \\ 1 & x_{21} & x_{22} \\ \vdots & \vdots & \vdots \\ 1 & x_{n1} & x_{n2} \end{bmatrix}$

- First column = all 1s (for the intercept c)
- Each row = one data point
- x_{ij} = value of the **j-th feature** for the **i-th example**

Produced Result:

$$\mathbf{w} = \begin{bmatrix} c \\ m_1 \\ m_2 \end{bmatrix}$$

$$y = m_1 x_1 + m_2 x_2 + c$$

The **first column of 1's** is **not the value of c**. It's a **placeholder column** to **learn C** (the **intercept**) during matrix multiplication.

These are the values that go into your final regression equation: $y = m_1 x_1 + m_2 x_2 + c$

$$y = m_1 x_1 + m_2 x_2 + \underbrace{c}_{\text{bias/intercept}}$$

Let’s see an example of math!

Linear Regression

Problem Statement

x_1 (sqft)	x_2 (bedrooms)	y (price)
1000	2	200000
1500	3	250000
2000	4	300000

Let's solve with matrix: $\mathbf{w} = (X^T X)^{-1} X^T y$

$$X = \begin{bmatrix} 1 & 1000 & 2 \\ 1 & 1500 & 3 \\ 1 & 2000 & 4 \end{bmatrix}, \quad y = \begin{bmatrix} 200000 \\ 250000 \\ 300000 \end{bmatrix}$$

$$X^T = \begin{bmatrix} 1 & 1 & 1 \\ 1000 & 1500 & 2000 \\ 2 & 3 & 4 \end{bmatrix}$$

Produced Result:

$$\mathbf{w} = \begin{bmatrix} c \\ m_1 \\ m_2 \end{bmatrix}$$

$$y = m_1 x_1 + m_2 x_2 + c$$

Correlation Analysis

Watch Tutorial

Correlation Analysis in machine learning (ML) is a statistical method used to measure the strength and direction of the relationship between two or more variables. It's a crucial step in **exploratory data analysis (EDA)** to understand how features in your dataset are related.

🔍 Why is Correlation Analysis Important in ML?

- **Feature Selection**: Helps identify which features are strongly related to the target variable.
- **Multicollinearity Detection**: Helps detect when two or more independent variables are highly correlated, which can affect model performance, especially in linear models.
- **Insights & Understanding**: Gives a clearer picture of relationships in your data.

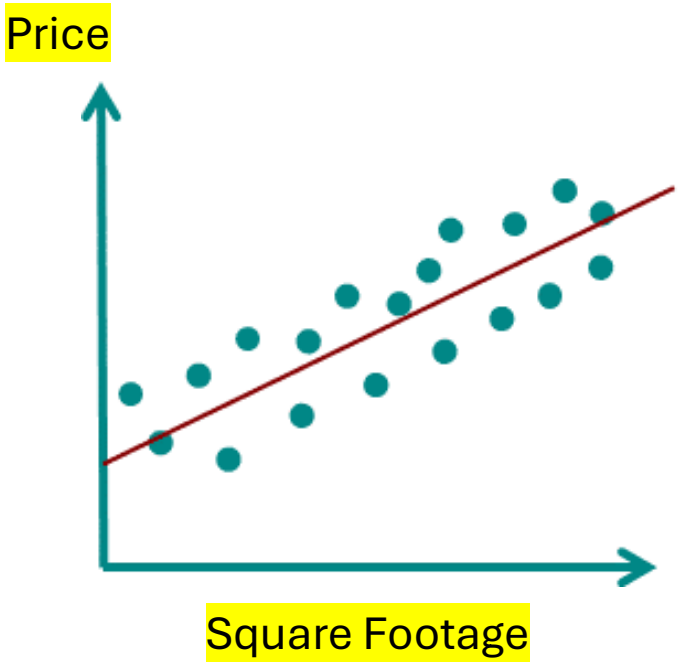
📊 Common Correlation Metrics

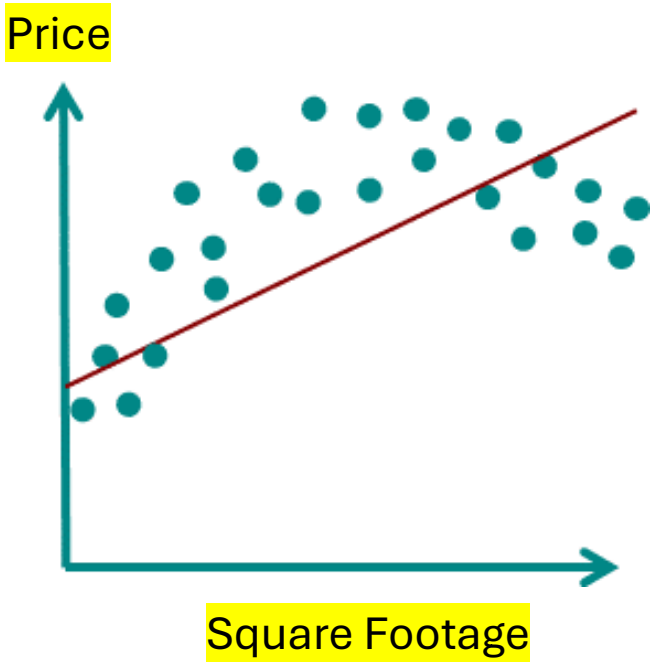
Metric	Best For	Range	Notes
Pearson	Linear relationships	-1 to 1	Most common; assumes normality
Spearman	Monotonic relationships	-1 to 1	Uses rank-order; non-parametric
Kendall Tau	Ordinal data	-1 to 1	More robust for small datasets

Coefficient (r)	Interpretation
1.0	Perfect positive correlation
0.7 to 0.9	Strong positive
0.4 to 0.6	Moderate positive
0.1 to 0.3	Weak positive
0	**No correlation**
-0.1 to -0.3	Weak negative
-0.4 to -0.6	Moderate negative
-0.7 to -0.9	Strong negative
-1.0	Perfect negative correlation

Correlation in Regression

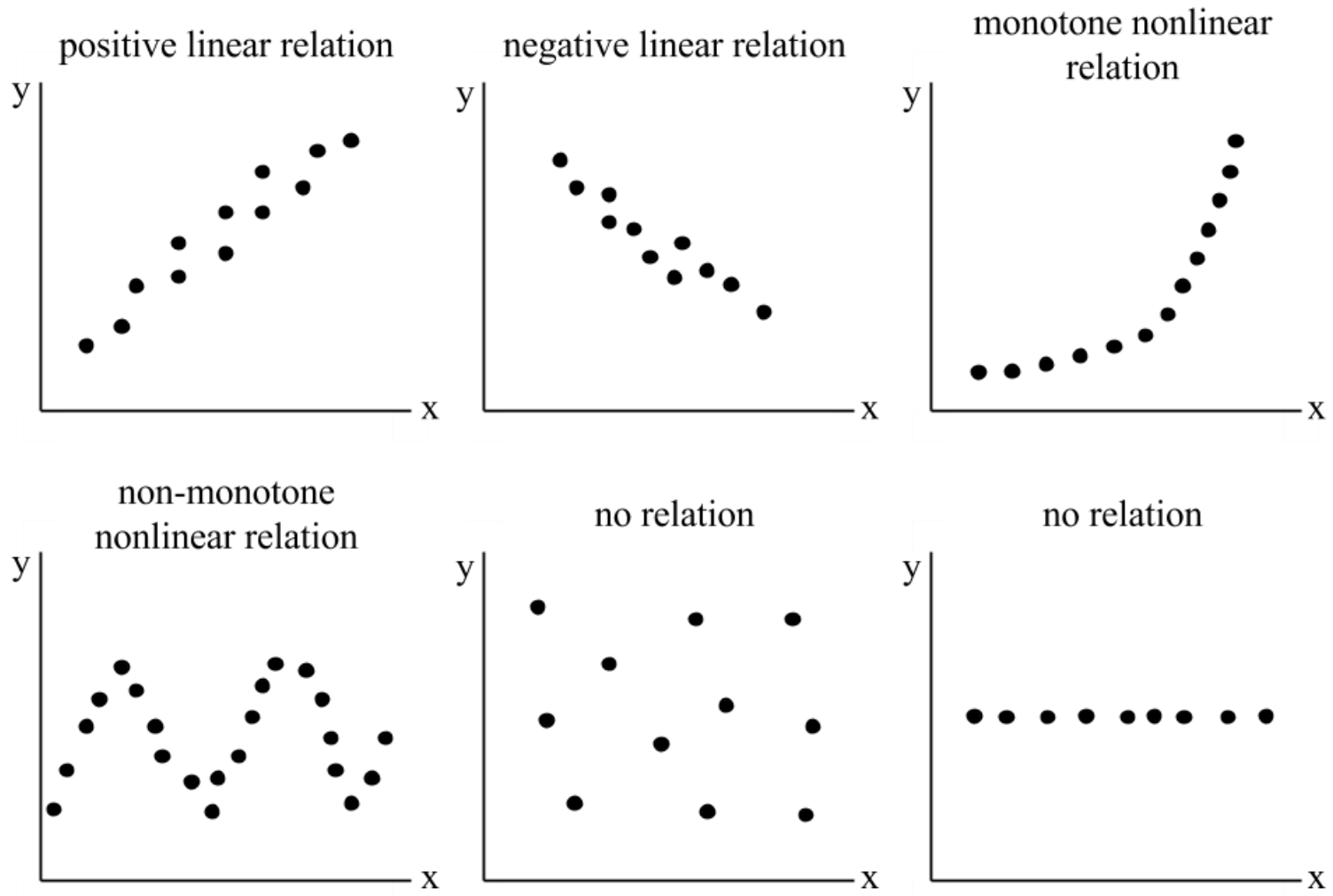
Machine Learning





Correlation in Regression

Machine Learning



Correlation in Regression

Pearson Correlation Coefficient

Pearson Correlation Coefficient: Correlation is always between two variables at a time. When we show a correlation matrix, it just lists all the pairwise correlations between every **pair of variables**.

$$r = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2} \cdot \sqrt{\sum(y_i - \bar{y})^2}}$$

Where:

- x_i, y_i are individual values of the two variables
- $\bar{x}, \bar{y}$ are their means
- r is the **correlation coefficient** (between -1 and 1)

Dataset

Sample	Size (x)	Price (y)	Age (z)
1	1400	250000	5
2	1600	300000	10
3	1800	350000	15

An example correlation table with **4 variables** commonly found in a housing dataset, showing how they might relate to House Price:

Feature	House Price	Size (sqft)	Number of Rooms	Age of House
House Price	1.00	0.88	0.75	-0.65
Size (sqft)	0.88	1.00	0.70	-0.40
Number of Rooms	0.75	0.70	1.00	-0.30
Age of House	-0.65	-0.40	-0.30	1.00

Interpretation: House Price has-

- a **strong positive** correlation with **Size (sqft)** (0.88)
- a **moderate positive** correlation with **Number of Rooms** (0.75)
- a **strong negative** correlation with **Age of House** (-0.65), meaning older homes tend to be cheaper.

Optimization: Gradient Descent

Watch Tutorial

Can I Perform Linear Regression using Gradient Descent?

Suppose your linear regression model is: $\hat{y} = w^T x + b$

Your loss function is typically **MSE**: $J(w,b) = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$

This cost function is **convex**, so gradient descent will converge to the **global minimum**

1. Compute the gradients:

$$\frac{\partial J}{\partial w} = \frac{2}{n}X^T(Xw + b - y)$$

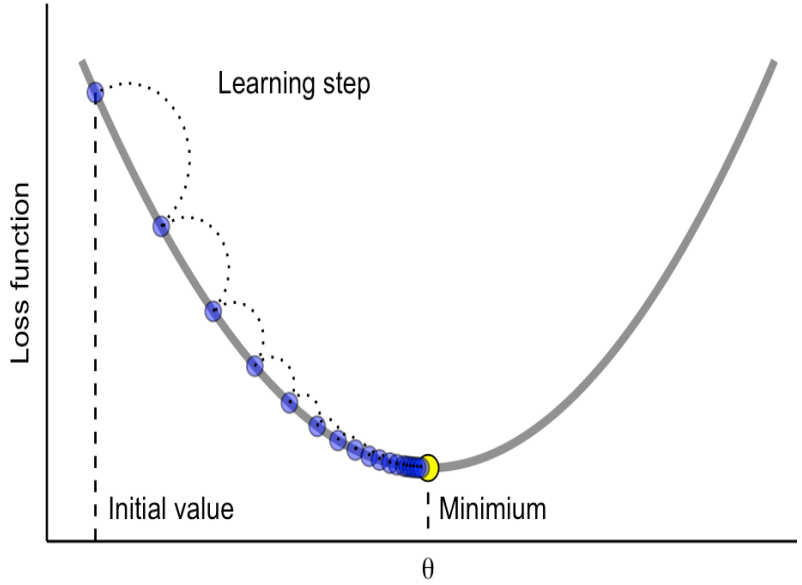
$$\frac{\partial J}{\partial b} = \frac{2}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)$$

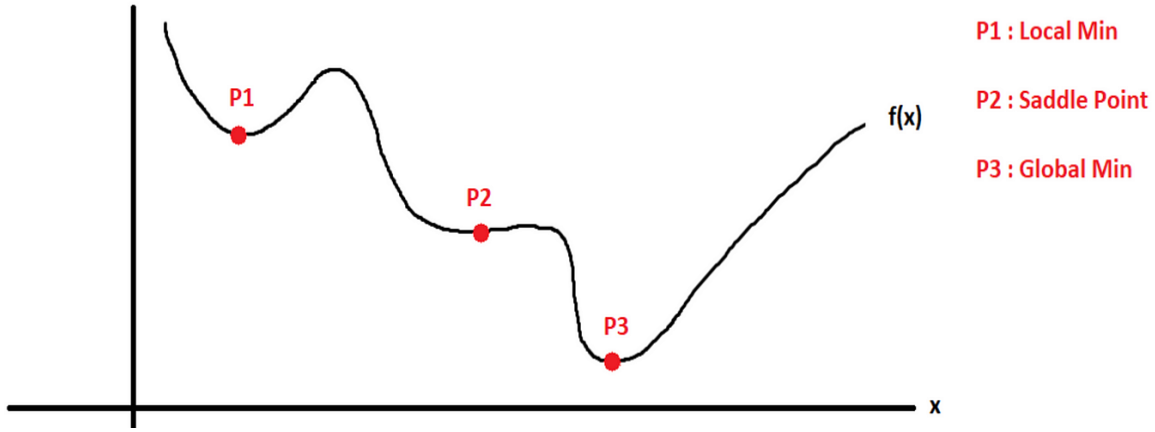
2. Update the parameters: $w := w - \alpha\frac{\partial J}{\partial w}$ $b := b - \alpha\frac{\partial J}{\partial b}$

Gradient Descent in ML & DL

Model Optimization

Gradient descent is a first-order **iterative optimization algorithm** for finding the minimum of a function. Gradient descent can be performed on any differentiable loss function. The main goal of gradient descent is to minimize a cost or loss function, optimizing model parameters for better performance.

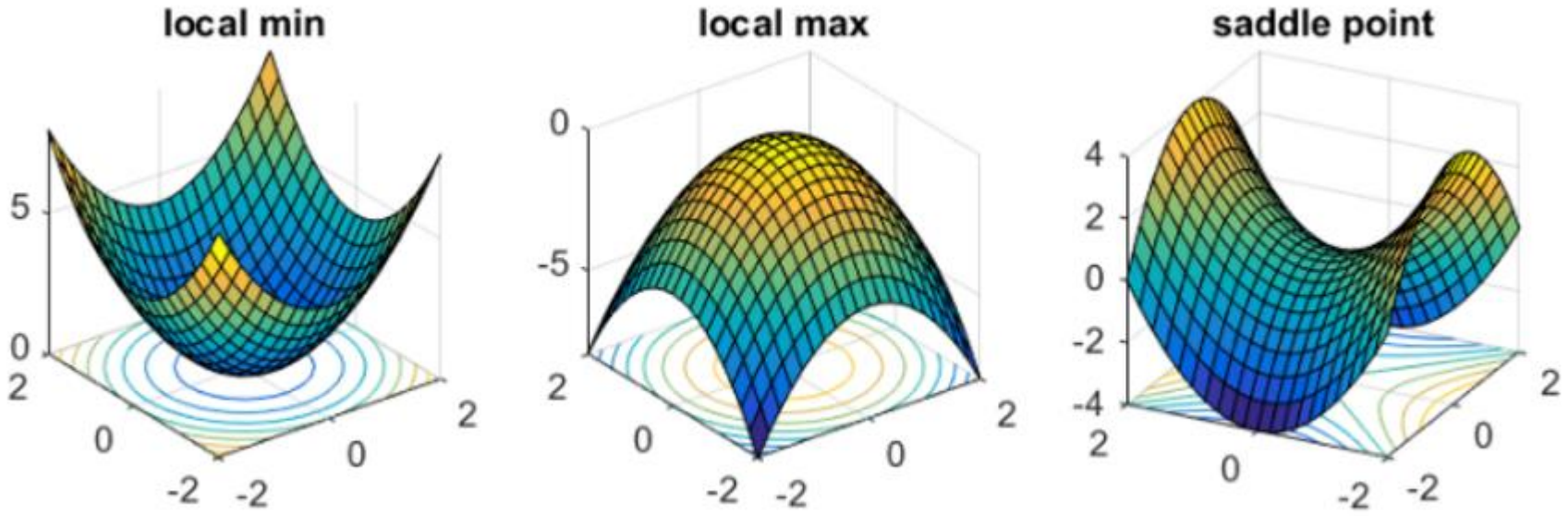




Convexity doesn't depend on MSE alone; It depends on the combination of the cost **function and the model**.

Gradient Descent in ML & DL

Model Optimization



The main equation for updating parameters in gradient descent is:

$$x_{\text{next}} = x - \alpha \cdot \nabla f(x) \quad \textbf{Or,} \quad w_{\text{new}} = w_{\text{old}} - \alpha \frac{\partial J}{\partial w}$$

where:

- x_{next} is the updated parameter value,
- x is the current parameter value,
- α is the learning rate,
- $\nabla f(x)$ is the gradient of the function f at x.

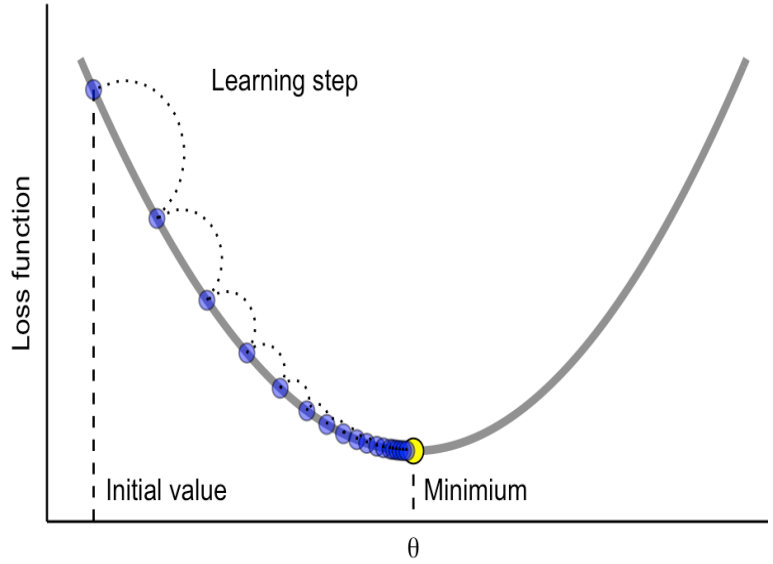


Fig: Gradient Descent

Gradient Descent in ML & DL

Model Optimization

$$w_{\text{new}} = w_{\text{old}} - \alpha \frac{\partial J}{\partial w}$$

where:

- w_{new} is the updated weight.
- w_{old} is the current weight.
- α is the learning rate.
- $\frac{\partial J}{\partial w}$ is the gradient of the loss function with respect to the weight.

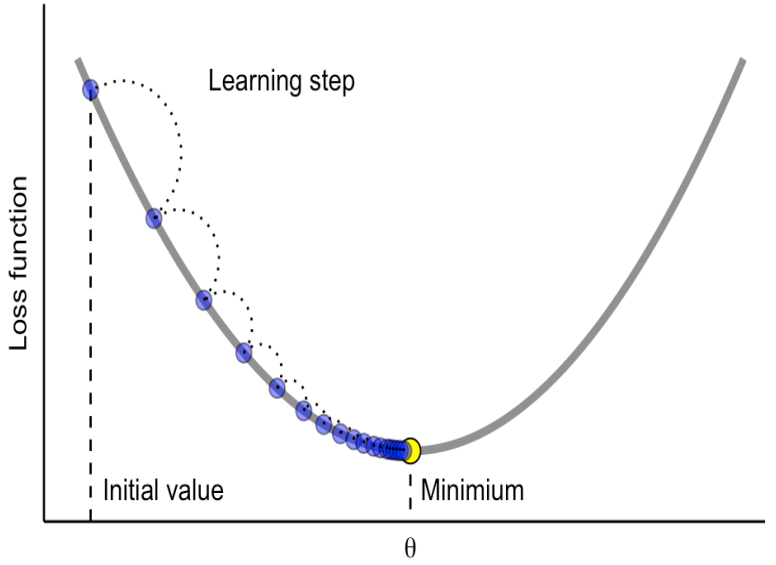


Fig: Gradient Decent

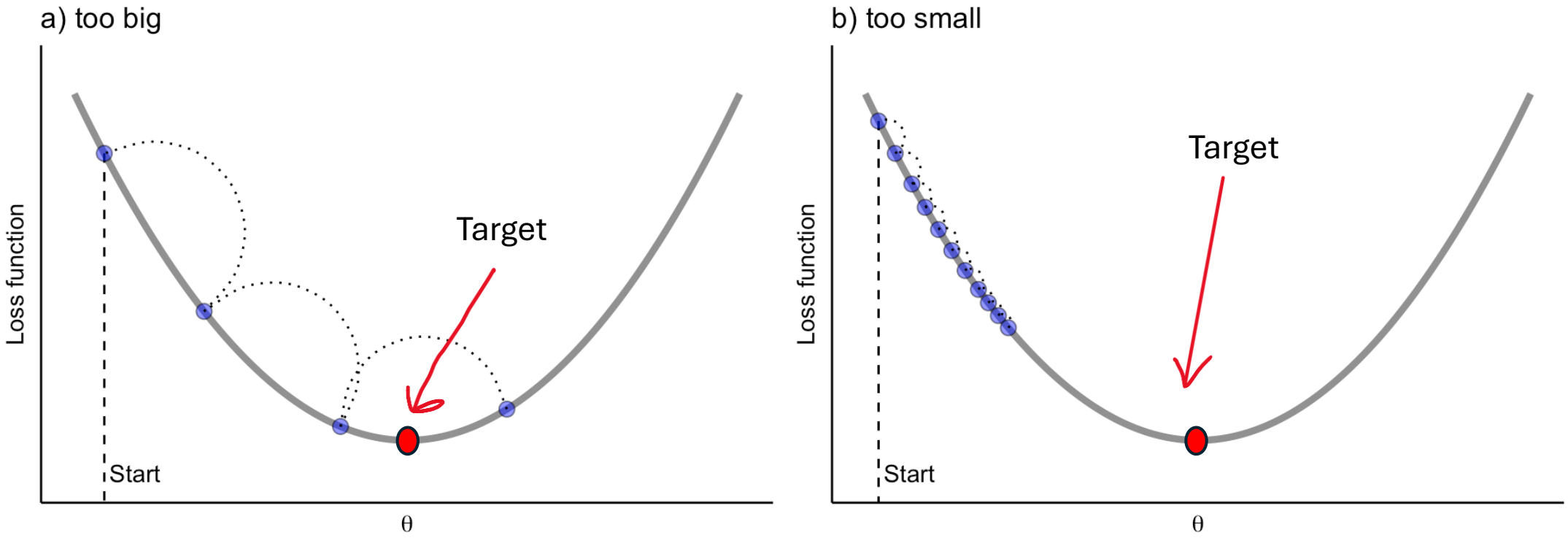
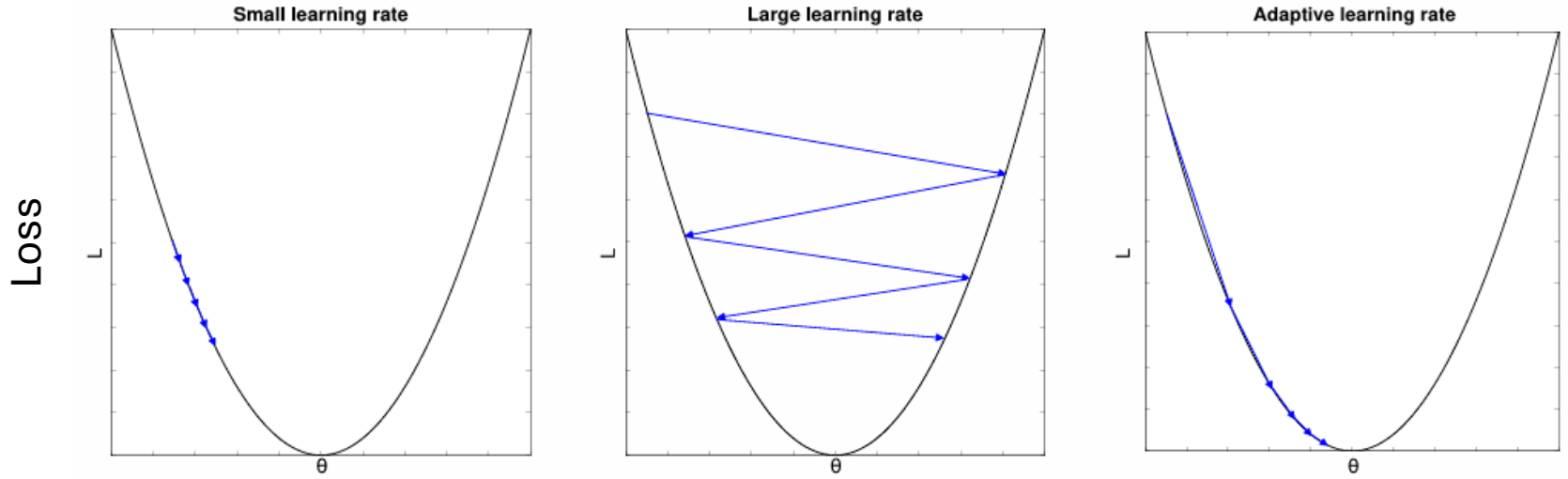


Fig: Importance of Learning Rate

Gradient Descent in ML & DL

Model Optimization



- η too small: long training time
- η too large: miss optima
- Practice: “learning rate decay”: adapt η gradually (e.g.: start with $\eta = 0.01$ and divide every *x* epoch by 10)

Variants of Gradient Descent

Watch Tutorial

Watch Tutorial

Polynomial Regression

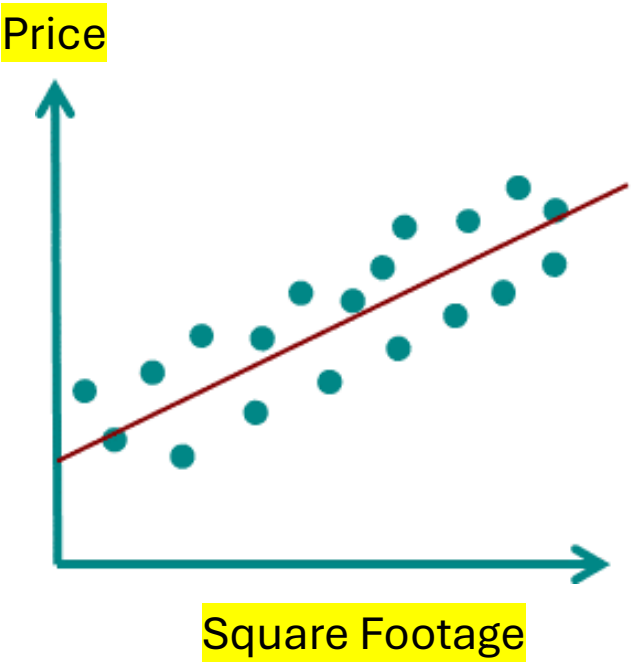
Watch Tutorial

Mean-Based Formula for OLS:

$$m = \frac{\sum(x - \bar{x})(y - \bar{y})}{\sum(x - \bar{x})^2}$$

$$c = \bar{y} - m\bar{x}$$

Price(y) = mx + c



Loss: Original_Price(y) - Price(y)

Why Do We Need **Polynomial Regression**?

Because sometimes, real-world data isn't a straight line.

And we need something a ***little curvier*** to capture its true pattern.

Polynomial Regression

Regression Analysis

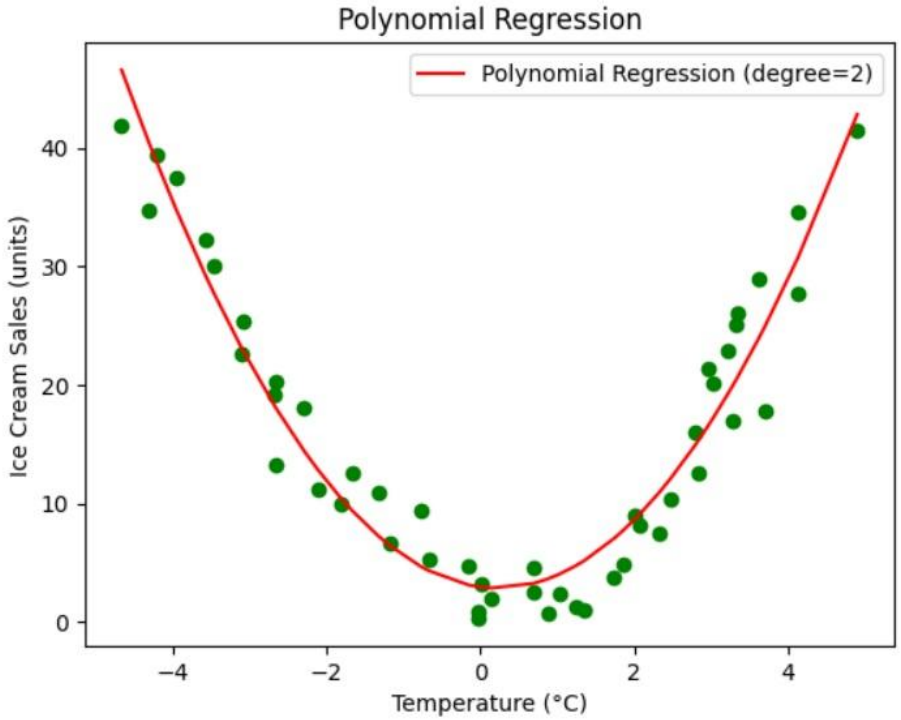
Polynomial regression is a **regression analysis** in which the **relationship between the independent variable x** and the **dependent variable y** is modeled as an **nth-degree polynomial**.

In simple terms:

It's a way to fit a **curved line** to data points, rather than just a straight line (like in linear regression).

When do you use it?

- When the data shows a **non-linear trend**.
- When linear regression doesn't fit the data well.



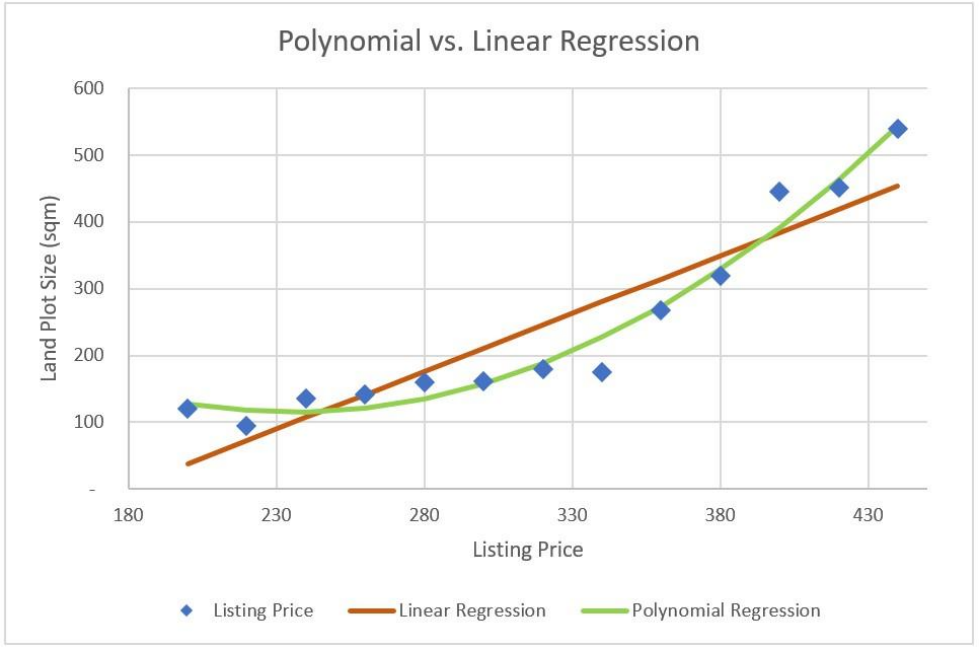
Polynomial regression is a **regression analysis** in which the **relationship between the independent variable x** and the **dependent variable y** is modeled as an **nth-degree polynomial**.

In simple terms:

It's a way to fit a **curved line** to data points, rather than just a straight line (like in linear regression).

When do you use it?

- When the data shows a **non-linear** trend.
- When linear regression doesn't fit the data well.



Polynomial regression is a **regression analysis** in which the **relationship between the independent variable x** and the **dependent variable y** is modeled as an **nth-degree polynomial**.

In simple terms:
It's a way to fit a **curved line** to data points, rather than just a straight line (like in linear regression).

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \cdots + \beta_n x^n + \varepsilon$$

Where:

- y: predicted value
- x: input variable
- $\beta_0, \beta_1, \ldots, \beta_n$: coefficients
- ε: error term
- n: the degree of the polynomial

The **degree** is the **highest power of the input variable(s)** in the polynomial function.

- Low degree → High bias, low variance (may miss patterns)
- High degree → Low bias, high variance (may overfit noise)

- 1st Degree: **Linear Regression:** $y = \beta_0 + \beta_1 x$

  Shape: Straight line

- 2nd Degree: **Quadratic Regression:** $y = \beta_0 + \beta_1 x + \beta_2 x^2$

  Shape: Parabola (U-shaped or ∩-shaped)

- 3rd Degree: **Cubic Regression:** $y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3$

  Shape: Complex curves with multiple bends

$$\text{Number of features} = \binom{n+d}{d} = \frac{(n+d)!}{d! \cdot n!}$$

Say you have n features and degree d; the number of generated features (including interactions and powers) is:

If You Have 1 Feature (univariate):

Degree	Total Features
1	1
2	2 (x, x^2)
3	3 (x, x^2, x^3)
4	4 (x, x^2, x^3, x^4)

$$\binom{1+1}{1} = \binom{2}{1} = 2$$

So, you get **2 terms**:

- One for the bias (constant term)
- One for the feature x

Polynomial Regression

Regression Analysis

$$\text{Number of features} = \binom{n+d}{d} = \frac{(n+d)!}{d! \cdot n!}$$

Degree (d)	Features (n)	Total Terms (with bias)	Generated Terms (examples)
1	1	$\binom{1+1}{1} = 2$	$1, x_1$
2	1	$\binom{1+2}{2} = 3$	$1, x_1, x_1^2$
2	2	$\binom{2+2}{2} = 6$	$1, x_1, x_2, x_1^2, x_1x_2, x_2^2$
3	2	$\binom{2+3}{3} = 10$	$1, x_1, x_2, x_1^2, x_1x_2, x_2^2, x_1^3, x_1^2x_2, x_1x_2^2, x_2^3$
3	3	$\binom{3+3}{3} = 20$	20 total combinations including all interactions and powers

Coefficient of Determination In Regression Analysis

Watch Tutorial

Say you want to predict the **price of a house** based on its **square footage**.

Square Footage (X)	Price ($Y)
1000	200000
1500	250000
2000	300000
2200 (Testing)	**350000**

Price = (Slope × Square Footage) + Intercept

Price(y) = mx + c

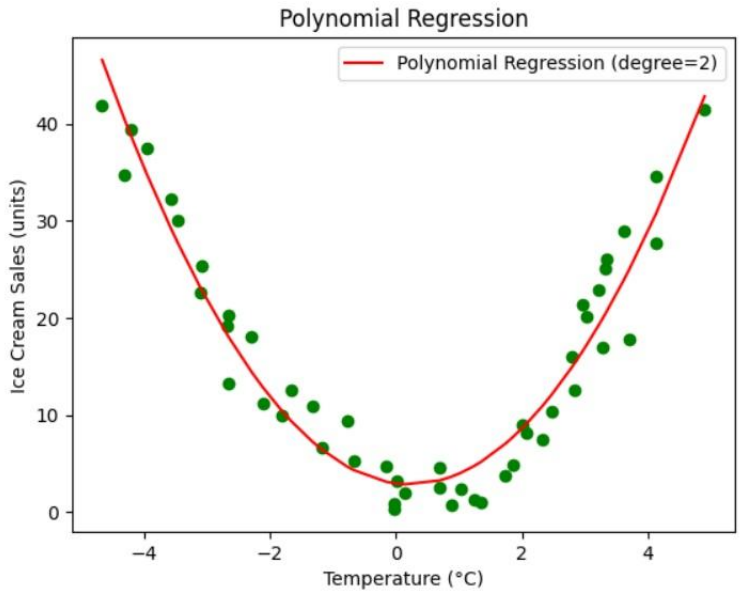
If your goal is to **predict price**, then the main objective is for your predicted values to be **as close as possible** to the **actual/original prices**.

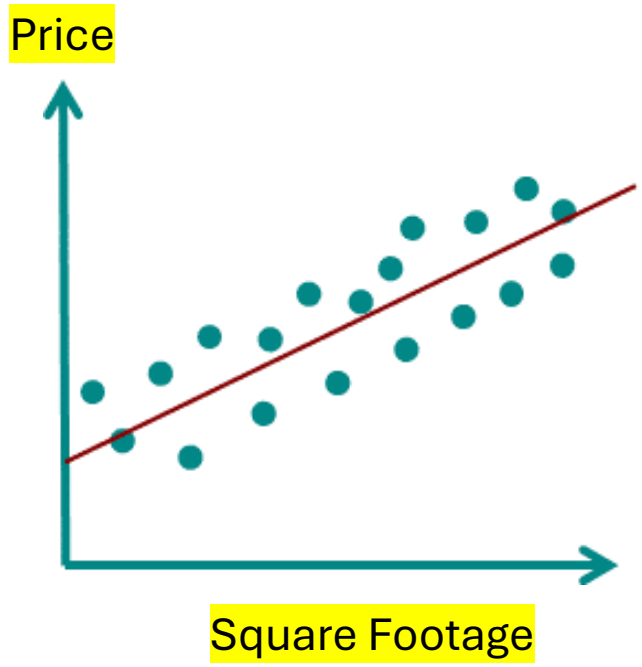
Coefficient of Determination

Regression Analysis

The **coefficient of determination**, often denoted as **R^2 (R-squared)**, is a statistical measure used to assess how well a regression model explains the variability of the dependent variable.

It tells you **how much of the variation in the output (Y)** can be **explained by the input features (X)** in a regression model. R^2 measures how well **any regression model,** including linear, polynomial, and neural network, explains the **variance in the target variable**, with 1 being a perfect fit. Range: $-\infty < R^2 \leq 1$





Coefficient of Determination

Regression Analysis

Formula: $R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$

- SS_{res} = Sum of Squares of Residuals
- SS_{tot} = Total Sum of Squares

Interpretation:

- **$R^2 = 1$**: Perfect fit. The model explains 100% of the variance.
- **$R^2 = 0$**: The model explains none of the variance—no better than just using the mean.
- **$R^2 < 0$**: Model is worse than just predicting the mean (can happen with bad models).

Cost Functions

Regression Analysis

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Logistic Regression

Watch Tutorial

Logistic regression is a statistical method for predicting the **probability of a certain outcome** based on one or more input variables. It's mainly used when the result you're predicting is **binary**, meaning it has two possible values (like yes/no, 0/1, good/bad, success/failure).

Probability= $$\frac{1}{1 + e^{-(\beta_0+\beta_1 x_1+\beta_2 x_2+\cdots+\beta_n x_n)}}$$

where:

- e is Euler's number (about 2.718),
- $\beta_0, \beta_1, \ldots$ are the parameters the model learns,
- $x_1, x_2, \ldots$ are the input features.

- Why it’s called Regression?
- Why it's still a Classification method?

Logistic regression is called "regression" because it models a relationship between input variables and a continuous outcome, but the continuous outcome it models is the probability of belonging to a class (between 0 and 1), not the class itself.

Logistic regression does **start** similarly to linear regression: It calculates a **linear combination** of the input features, like:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

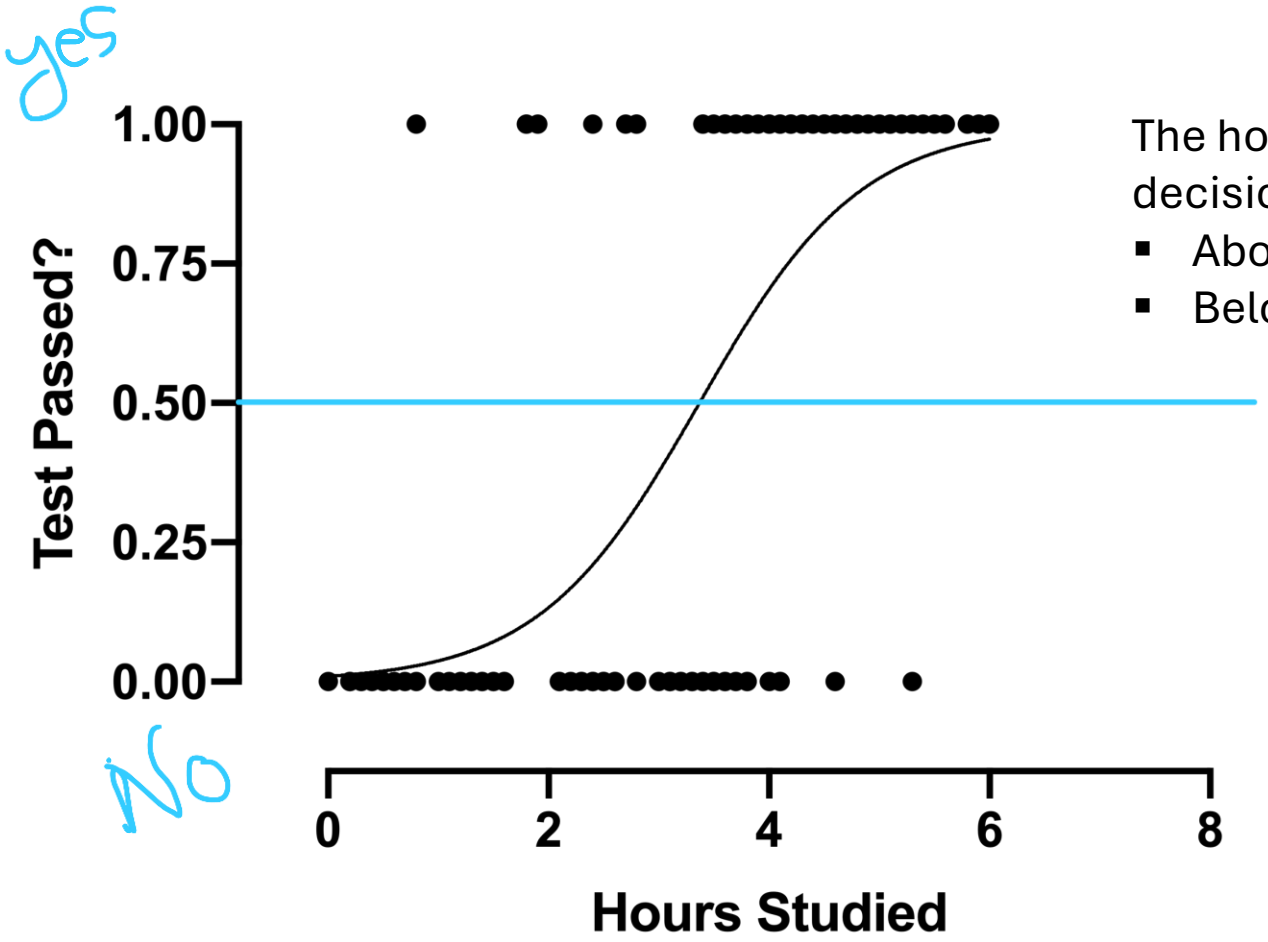
This part is **linear**, just like linear regression.

❌ **It doesn't stop there.**

Instead of outputting z directly (which could be any number from −∞ to +∞), logistic regression **plugs z into a sigmoid (logistic) function** to produce a value between 0 and 1:

$$\text{Output} = \frac{1}{1 + e^{-z}}$$

- Linear regression → direct numeric output (no squashing).
- Logistic regression → linear combination PLUS sigmoid to get a probability.



The horizontal **blue line** at 0.5 is the decision boundary:

- Above 0.5 → Predict "Yes" (pass).
- Below 0.5 → Predict "No" (fail).